

R08

GUIADO EN CLASE

Exportación a CSV/JSON y Normalización de datos

Programación para Ciberseguridad · Fase I · AD2026

CSV es el formato más universal para compartir datos tabulares (lo abre Excel, cualquier herramienta de análisis). JSON es el formato preferido cuando otro programa consumirá los datos. Normalizar antes de exportar evita errores de comparación posteriores.

Objetivo: Exportar un inventario a CSV y JSON, y construir una propiedad calculada que normalice los datos antes de exportarlos.

Instrucciones

1. Exportar a CSV y JSON:

```
Get-Process | Select-Object Name, CPU, Id |  
    Export-Csv -Path "$env:USERPROFILE\Desktop\procesos.csv" -NoTypeInfo  
Get-Process | Select-Object Name, CPU, Id | ConvertTo-Json |  
    Out-File -FilePath "$env:USERPROFILE\Desktop\procesos.json"
```

2. Abre ambos archivos con el Bloc de notas y observa la diferencia de estructura.
3. Normalizar con propiedad calculada:

```
Get-LocalUser |  
    Select-Object @{Name='Usuario'; Expression={$_.Name.Trim().ToLower()}}, Enabled  
Get-Date -Format "yyyy-MM-dd HH:mm:ss"
```

4. Reporte normalizado final:

```
$reporte = Get-Process | Select-Object '  
    @{Name='Proceso'; Expression={$_.Name.ToLower()}},  
    CPU,  
    @{Name='Fecha'; Expression={Get-Date -Format 'yyyy-MM-dd'}}  
$reporte | Export-Csv -Path "$env:USERPROFILE\Desktop\reporte_normalizado.csv" -  
    NoTypeInfo
```

Evidencias requeridas

- Screenshot de `procesos.csv` y `procesos.json` abiertos, mostrando la diferencia de formato.
- Screenshot del reporte normalizado generado.

Preguntas de reflexión

1. Sin normalizar, "Admin", "admin" y "ADMIN" serían tres usuarios distintos al comparar. ¿Qué tipo de error concreto podría aparecer en un reporte de auditoría si no normalizas antes de contar?
2. ¿Por qué JSON es preferible a CSV cuando otro programa va a consumir los datos automáticamente (por ejemplo, un SIEM)?
3. El parámetro `-NoTypeInfo` evita una línea extra en el CSV. Abre el archivo sin ese parámetro y observa qué aparece. ¿Por qué esa línea extra podría romper un análisis automatizado?

R09

GUIADO EN CLASE

Tu propio Blue Team Cookbook

Programación para Ciberseguridad · Fase I · AD2026

Un *cookbook* defensivo es una colección de funciones ya probadas que un equipo de seguridad reutiliza sin reescribir el mismo código de auditoría cada vez. Es la aplicación práctica de todo lo aprendido en Fase I.

Objetivo: Construir el módulo **Cookbook** con tres funciones reutilizables que integren inventario, detección de eventos y exportación normalizada.

Instrucciones

1. Crea la carpeta y el archivo del módulo:

```
New-Item -Path "$env:USERPROFILE\Documents\WindowsPowerShell\Modules\Cookbook" -
    ItemType Directory
New-Item -Path "$env:USERPROFILE\Documents\WindowsPowerShell\Modules\Cookbook\
    Cookbook.psm1" -ItemType File
```

2. Escribe las tres funciones dentro del `.psm1`:

```
function Get-InventarioRapido {
    Get-CimInstance -ClassName Win32_ComputerSystem | Select-Object Name,
    Manufacturer, Model
    Get-CimInstance -ClassName Win32_LogicalDisk | Select-Object DeviceID, Size,
    FreeSpace
}
function Find-LoginsFallidos {
    param ([Parameter(Mandatory=$false)] [int]$Maximo = 5)
    Get-WinEvent -LogName Security -MaxEvents 2000 -ErrorAction SilentlyContinue |
        Where-Object { $_.Id -eq 4625 } | Select-Object -First $Maximo TimeCreated,
    Id
}
function Export-ReporteSeguro {
    param (
        [Parameter(Mandatory=$true)] $Datos,
        [Parameter(Mandatory=$true)] [string]$NombreArchivo
    )
    try {
        $Datos | Export-Csv -Path $NombreArchivo -NoTypeInfo -ErrorAction
        Stop
        Write-Output "Reporte guardado en: $NombreArchivo"
    }
    catch { Write-Warning "No se pudo exportar: $($_.Exception.Message)" }
}
```

3. Genera el manifiesto, importa y usa el módulo:

```
New-ModuleManifest -Path "...Cookbook.psd1" -RootModule 'Cookbook.psm1' -Author '
    TuNombre' -ModuleVersion '1.0' -Description 'Blue Team Cookbook'
Import-Module Cookbook
Get-InventarioRapido
Find-LoginsFallidos -Maximo 3
```